

Rebhi Mohamed Aziz

+216 92048665 Mohamedaziz.rebhi@ensi-uma.tn [azizrebhi](#) [aziz-rebhi](#)

Summary

Final-year Computer Science Engineer with hands-on experience building **scalable Python backend systems** using **PostgreSQL, Redis, and Docker**. Passionate about **data pipelines, API design, and distributed systems**. Strong focus on **code quality, testing (76% coverage)**, and **open-source contribution (15 merged PRs)**. **Quick and autonomous learner** with production experience in high-throughput systems and asynchronous architectures.

Education

National School of Computer Science (ENSI)

Engineering Degree in Computer Science

2023 – Sep 2026

Manouba, Tunisia

Preparatory Institute for Engineering Studies of Monastir

Preparatory Cycle in Mathematics and Physics

2021 – 2023

Monastir, Tunisia

Experience

Software Developer Intern — AI Infrastructure

Sofrecom Tunisia

Feb 2026 – Apr 2026

Tunisia

- Architected **production RAG pipeline** serving 1000+ queries/day using FastAPI, LangGraph, pgvector; reduced latency **40%** through bi-encoder reranking and improved accuracy **28%** via RAGAS metrics optimization.
- Containerized multi-service architecture with **Docker Compose** managing 5 services; automated deployment reducing setup time from 2 hours to **5 minutes**.

Software Developer Intern — Local LLM & MCP Integration

Sofrecom Tunisia

Jul 2025 – Aug 2025

Tunisia

- Deployed **DeepSeek LLM locally via Ollama** on 8GB RAM achieving **200ms latency** and **60% memory reduction** through quantization.
- Engineered **PostgreSQL MCP server** for natural language to SQL; integrated with GitHub Copilot/Claude Desktop supporting 15+ patterns with **95% accuracy** and **99.2% uptime**.

Open Source Contributions

OWASP BLT — 15 merged PRs across 4 repos; implemented **PATCH /{id}** endpoint using **class-based architecture with inheritance** following ORM patterns, JWT auth, role-based authorization, and **18 test cases (100% coverage)**.

OWASP LLM Top 10 (2026) — Sprint 1 researcher on LLM08 Vector Embedding Weaknesses; identified “Membership Inference via Similarity Search” attack, incorporated as Common Example 5 in the 2026 official draft alongside security leads.

Key Projects

DevLog | *FastAPI, PostgreSQL, Redis, Celery, Docker, GitHub Actions, pytest* | [GitHub](#)

- Built from scratch demonstrating **full ownership of architectural decisions**: cursor-based pagination over offset for consistency under load, **cache-aside pattern** for idempotent reads, **Alembic migrations** for zero-downtime schema changes, and async SQLAlchemy ORM with proper session management.
- Implemented **object-oriented design** with model inheritance, service layer abstraction, and dependency injection; deployed **5-service system** (FastAPI, PostgreSQL, Redis, Celery worker/beat) with comprehensive error handling and structured logging.
- Established rigorous **quality standards: 19 automated tests with 76% coverage**, CI/CD pipeline on every push, integration tests with PostgreSQL/Redis, and code refactoring following SOLID principles.
- Built **async data pipeline** using Celery Beat processing 1000+ records/week with exponential backoff retry; **Redis cache-aside** reduced endpoint latency **90%** (2100ms → 120ms) on high-frequency read paths; PostgreSQL indexing on high-cardinality columns supporting analytics queries at scale.

ARE-Nexus | *React, FastAPI, ChromaDB, Supabase* | [Live](#)

- Built **React frontend** with component architecture, state management (Context API), and async data fetching; **FastAPI backend** with async endpoints, PostgreSQL integration via Supabase, and real-time updates for RoboCup competition management serving 200+ users.

Technical Skills

Languages: Python, SQL, JavaScript, TypeScript

Backend & APIs: FastAPI, REST APIs, async/await, event-driven architecture, WebSockets

Databases: PostgreSQL (SQLAlchemy ORM, query optimization, indexing, Alembic migrations), Redis (caching, pub/sub), pgvector

Data & Pipelines: Celery (task queues, scheduled jobs, retry logic), background workers, batch processing, async data ingestion

Infrastructure: Docker, Docker Compose, GitHub Actions CI/CD, containerization, service orchestration

Testing & Quality: pytest, pytest-asyncio, integration testing, TDD, 76% coverage, SOLID principles

Collaboration: Git, GitHub (15 merged PRs), code review, open-source contribution